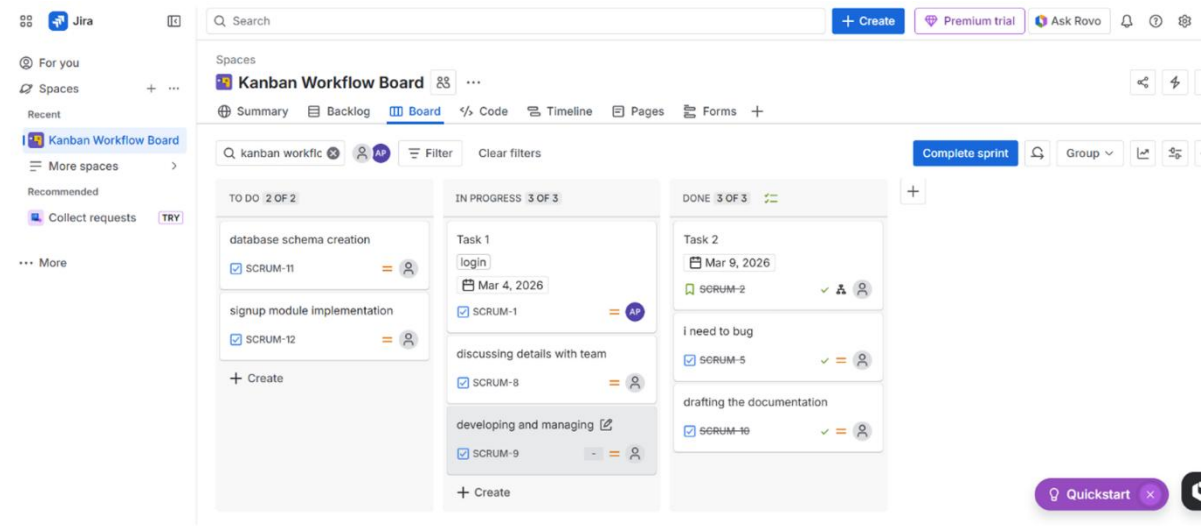
[CSA1011] - SOFTWARE ENGINEERING

NAME : DANIEL SAM K

REG NO : 192524081

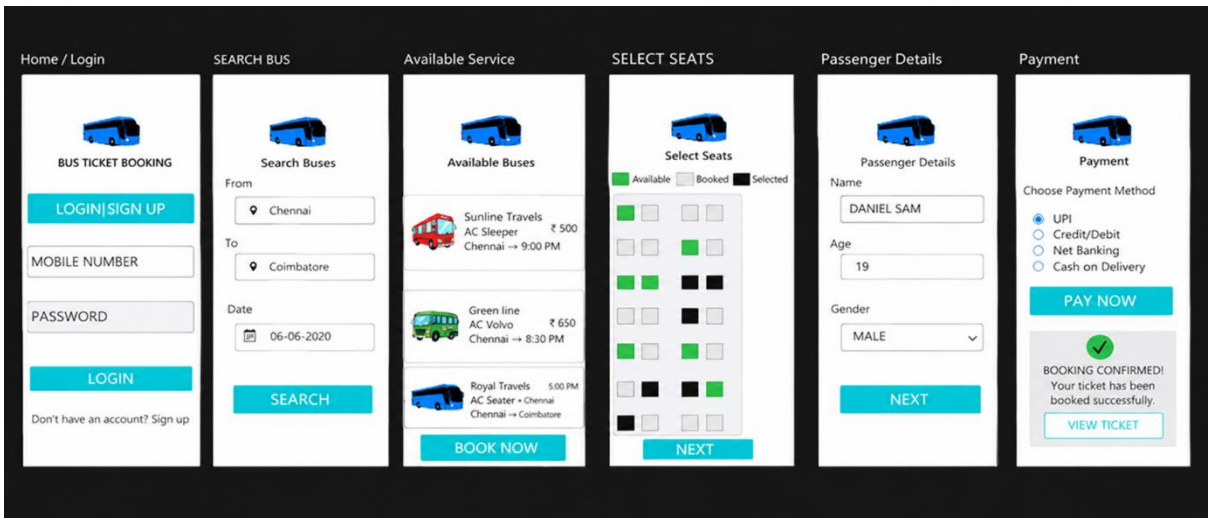
1. Create a Kanban Board to Visualize the Tasks.

- Create Columns for To Do, In-Progress and Done.
- Add Atleast 5 Sample Tasks
- Move the Tasks across the Columns to Simulate the Workflow.

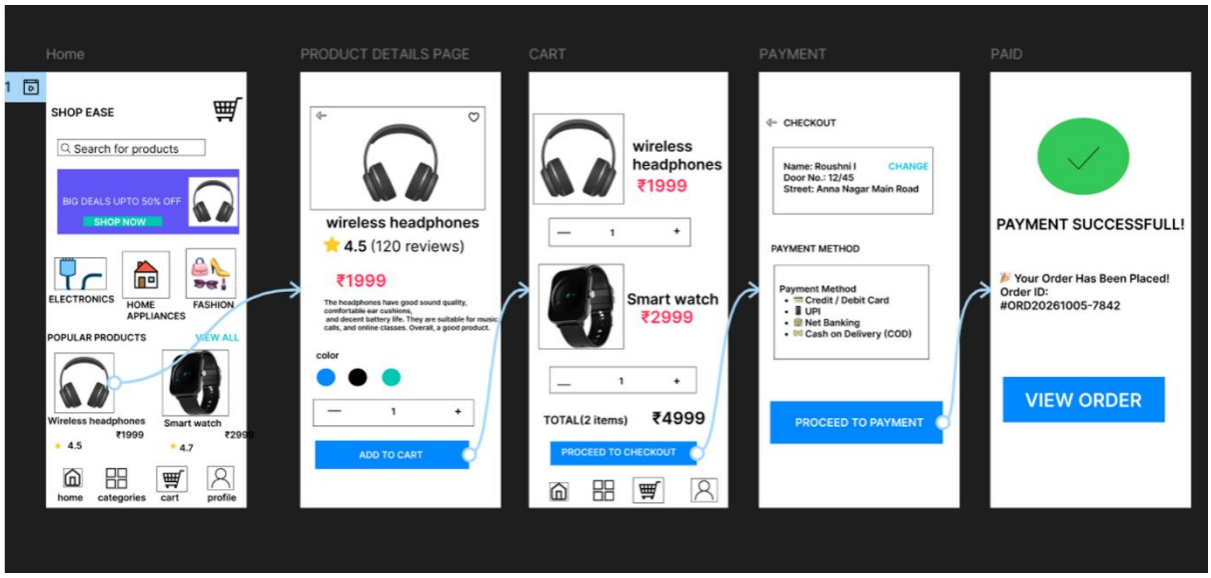


2. Sketch a Simple Prototype of a Bus Ticket Booking System using

Figma Tool

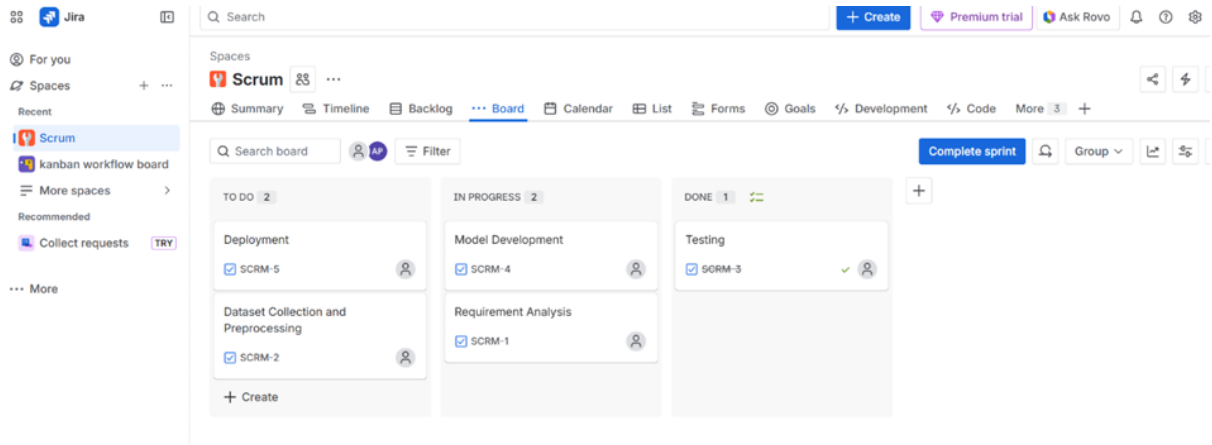


3. The stakeholders have conflicting views on the user interface design for an E-Commerce mobile app. Create a prototype using Figma tool to discuss with the stakeholders to get their feedback and approval.



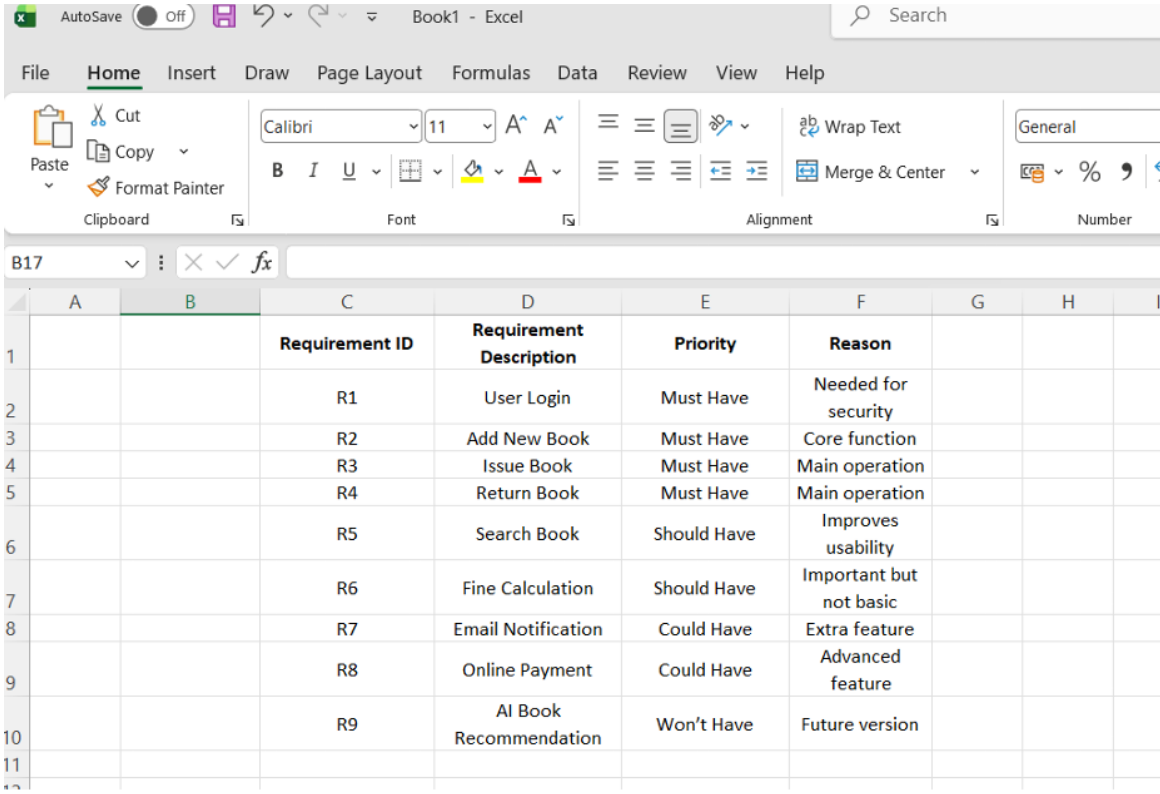
4. Create a Scrum Project in Jira.

- Add a backlog with at least 5 items (e.g., "Create user
- registration page", "Develop API for login").
- Prioritize the backlog and create a 1-week sprint.
- Move backlog items into the sprint and start the sprint.
- Finally show the Screenshot of the sprint board at the start and
- end of the sprint.



5. Use the following requirements for a Library Management System:

- Add a feature to search books by title and author.
- Implement an online book reservation system.
- Generate monthly reports on borrowed books for
- administrators.
- Enable email notifications for overdue books.
- Add support for QR code scanning for borrowing and
- returning books.
- Create a user-friendly dashboard for librarians.
- Allow users to review and rate books.
- Integrate a chatbot for user assistance.
- Develop a mobile app version of the system.
- Provide multi-language support.
- Categorize each requirement using MOSCOW Method (Must-Have,
- Should-Have, Could-Have, or Won't-Have) based on the following
- criteria:
- Impact on the users and stakeholders.
- Feasibility considering time, budget, and resource constraints.
- Finally Submit the completed Google Sheet or Excel file with all
- requirements categorized and justified.

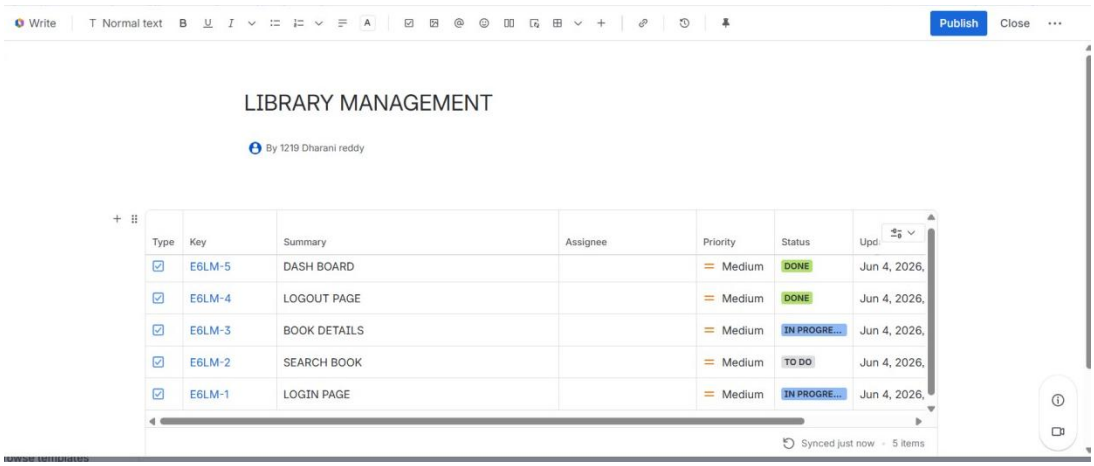


	A	B	C	D	E	F	G	H
1			**Requirement ID**	**Requirement Description**	**Priority**	**Reason**		
2			R1	User Login	Must Have	Needed for security		
3			R2	Add New Book	Must Have	Core function		
4			R3	Issue Book	Must Have	Main operation		
5			R4	Return Book	Must Have	Main operation		
6			R5	Search Book	Should Have	Improves usability		
7			R6	Fine Calculation	Should Have	Important but not basic		
8			R7	Email Notification	Could Have	Extra feature		
9			R8	Online Payment	Could Have	Advanced feature		
10			R9	AI Book Recommendation	Won't Have	Future version		
11								

6. Link Jira tasks with Confluence to streamline task tracking and progress monitoring for the Library Management System development.

Create a new page in Confluence titled "Library Management

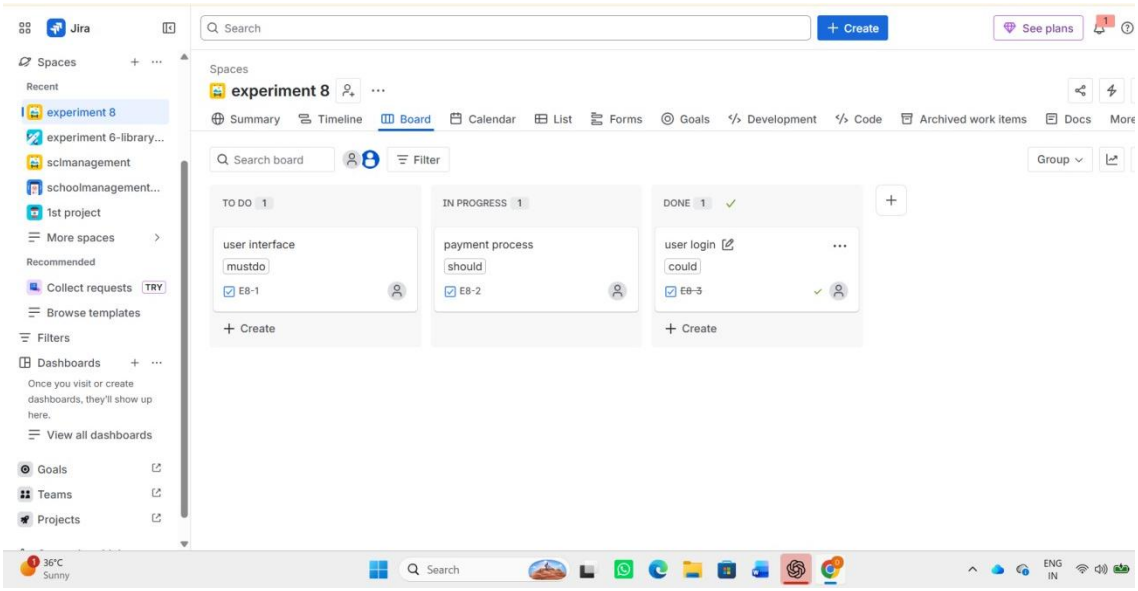
- System Project Overview."
- Embed at least 5 Jira issues related to the development of the
- Library Management System (e.g., tasks from the sprint like
- "Develop book search functionality," "Create user login page,"
- etc.).
- Use the Jira macro to display issues with status (e.g., "To Do,"
- "In Progress," "Done").
- Add a progress bar in the Confluence page to visually track
- the completion of each embedded Jira task (e.g., percentage of
- tasks completed in the sprint).
- Submit a screenshot of the Confluence page showing the
- embedded Jira tasks and the progress bar.



7. You are designing a Task Management System for a small team. The system should include the following features:

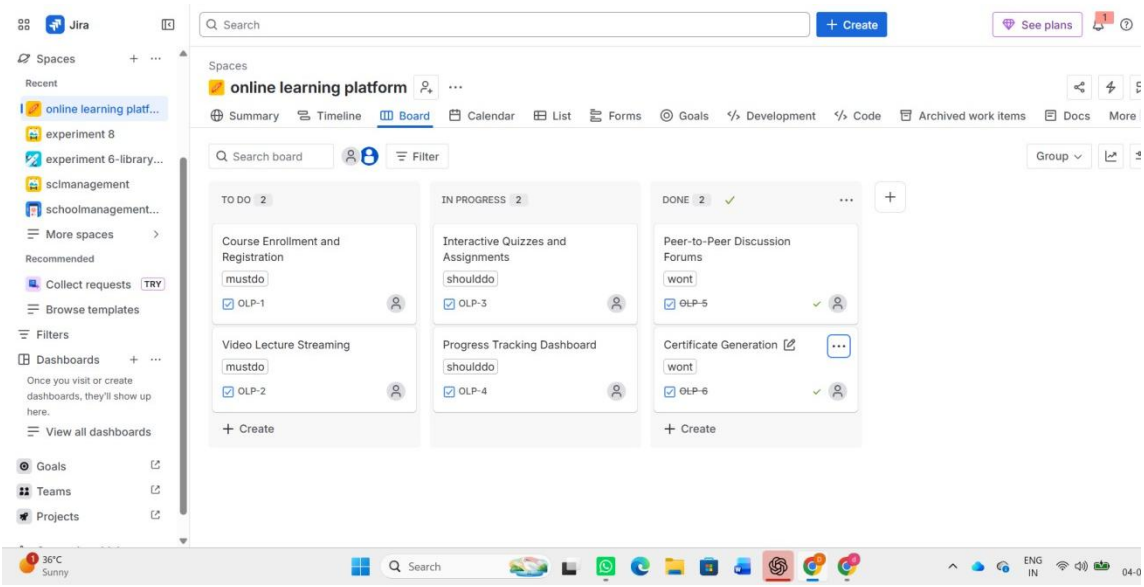
- 1.User Login and Role Assignment
- 2.Task Creation and Assignment
- 3.Task Prioritization and Deadlines
- 4.Progress Tracking and Reporting

- Prioritize these requirements using the MoSCoW and Kano models in
- Jira.



8. You are tasked with developing an Online Learning Platform. The platform should include the following functionalities:

- Course Enrollment and Registration
- Video Lecture Streaming
- Interactive Quizzes and Assignments
- Progress Tracking Dashboard
- Peer-to-Peer Discussion Forums
- Certificate Generation
- Use Jira to categorize and prioritize these requirements using
- MoSCoW and Kano techniques.

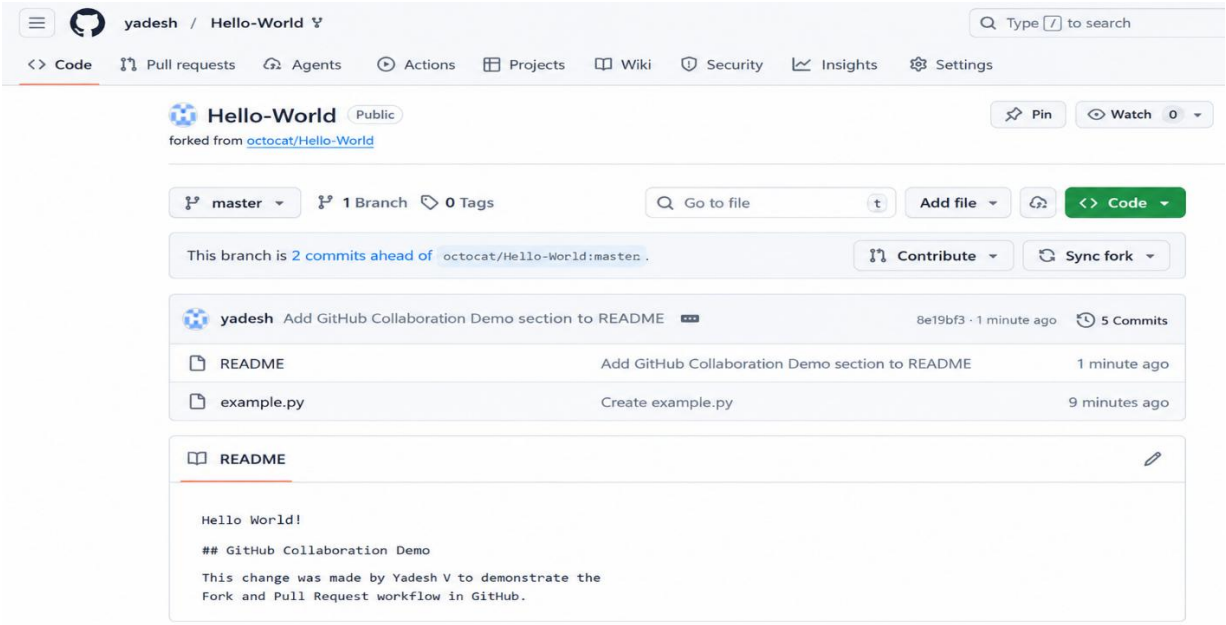


9. Demonstrate how to work collaboratively in Git/GitHub on a project

using the fork-and-pull request workflow.

Tasks:

- Fork an existing public GitHub repository (e.g., a sample
- JavaScript or Python project).
- Clone the forked repository to your local machine using Git.
- Create a new branch for the feature or change you want to
- work on.
- Make modifications or add new features (e.g., add a function,
- fix a bug, or update the README).
- Commit your changes and push the branch to GitHub.
- Go to the GitHub repository and create a pull request to merge
- your feature branch into the main branch.
- Review the pull request and provide feedback on the changes.
- Respond to feedback by making additional commits to the feature branch if necessary.
- Once the pull request is approved, merge it into the main
- branch.
- Finally submit the link to the pull request along with a
- summary of the changes you made and how you collaborated.



10. Demonstrate how to work with Git branches and resolve merge conflicts when collaborating with others.

Tasks:

- Clone a shared repository to your local machine.
- Create a new branch and switch to it.
- Make changes to a file (e.g., update a README or modify
- code in a specific function).
- Commit your changes.
- Push the changes to the remote repository.
- Before merging, pull the latest changes from the main branch.
- Switch back to your feature branch.
- Merge the main branch into your feature branch.
- If there are conflicts, resolve them manually by editing the
- conflicting files. After resolving conflicts, mark the conflicts
- as resolved.
- Commit the resolved merge.
- Push your feature branch with the merged changes to GitHub
- and create a pull request.
- Submit a summary of the steps you performed, the conflicts
- you encountered, and how you resolved them.

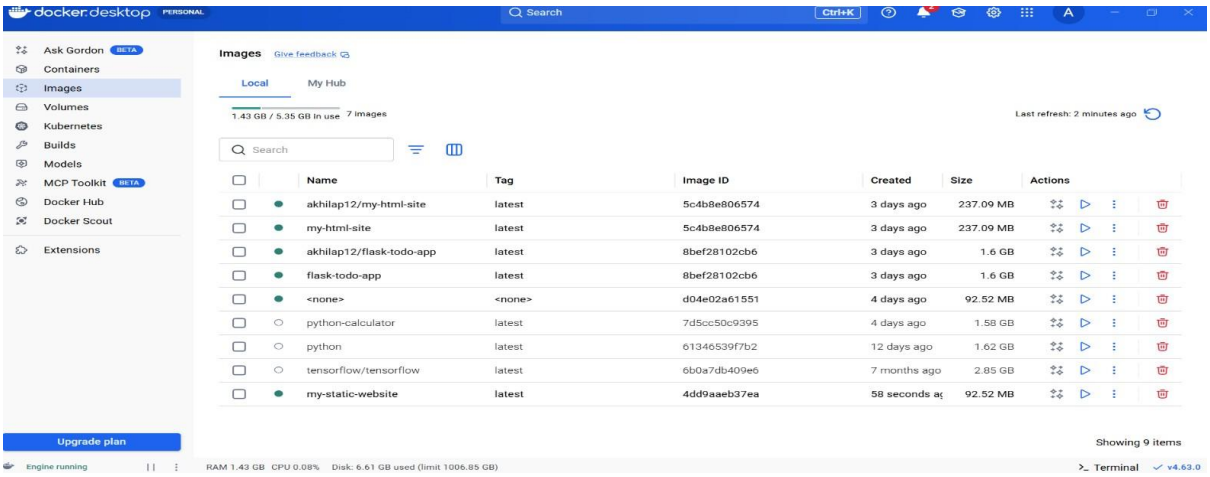


11.Create a Static Website and Containerize, Build & Serve it using

Docker.

Tasks:

- Create a Simple Static Website (index.html file) with basic
- HTML content.
- Write/create a Dockerfile to serve the website using Nginx.
- Build the Docker Image
- Run the container:
- Access the Website using a Browser

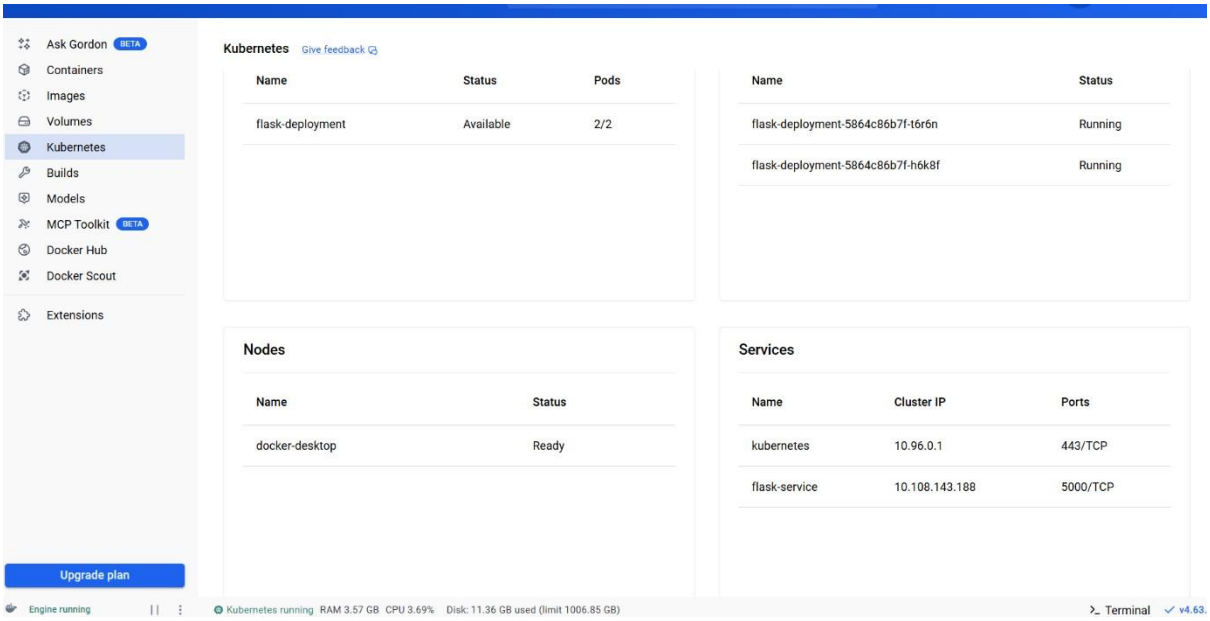


12. Create a Simple Python Flask API, Containerize the Application, Build & Push the Image using Docker and Deploy the Application using Kubernetes.

Tasks:

- Create a Simple Flask API by writing a Python file (app.py)
- with basic endpoints.
- Containerize the Flask App using Dockerfile.
- Build the Image using Docker.
- Push the Image to Docker Hub.
- Create Kubernetes manifests (Deployment YAML & Service
- YAML) to deploy the application.

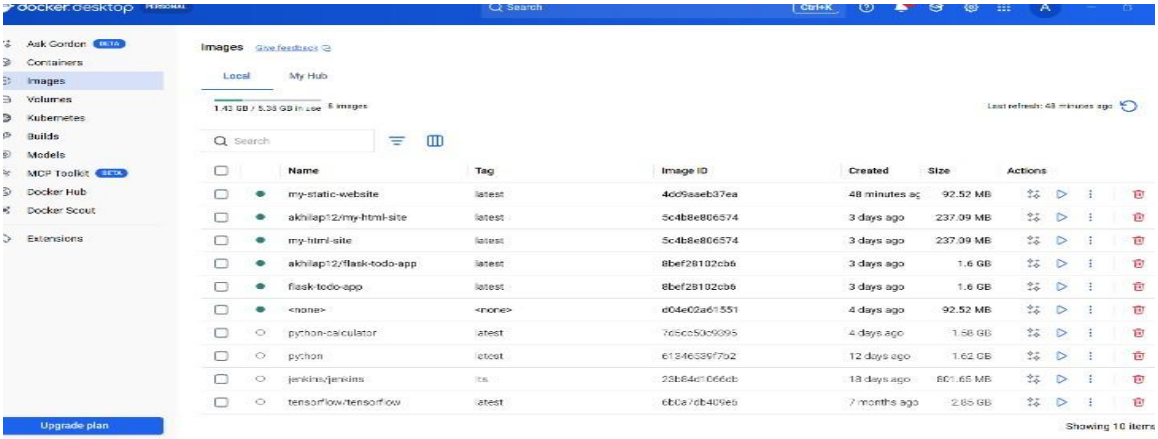
- Apply the Manifests and Access the API via NodePort.



13. Set up a CI/CD pipeline to automate the building, testing, and deployment of a containerized application.

Tasks:

- Set up Jenkins on a local machine or server.
- Create a Dockerfile to containerize a sample application.
- Write a Jenkinsfile to automate the process of building the Docker container, running tests, and deploying to a cloud platform (e.g., AWS or GCP).
- Configure Jenkins to trigger builds upon code commits or pull requests.

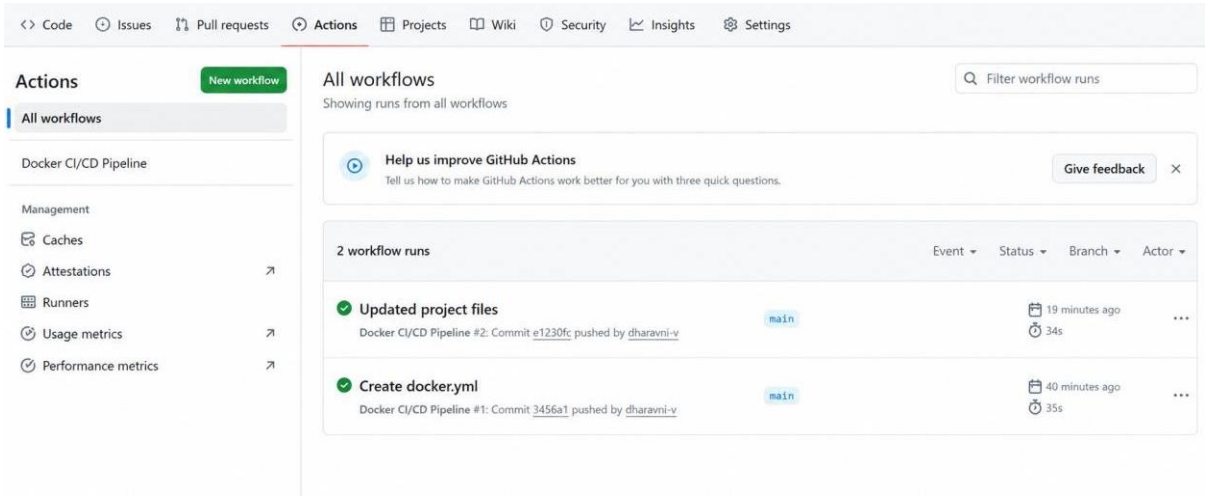




14. Implement Continuous Deployment using GitHub Actions to deploy a Dockerized application.

Tasks:

- Set up a GitHub repository and push a simple Dockerized
- application.
- Create a GitHub Actions workflow to automatically build and
- push the Docker image to Docker Hub or GitHub Container
- Registry.
- Automate deployment to a cloud platform (e.g., AWS ECS,
- Azure Kubernetes Service, or Google Kubernetes Engine).
- Test the CI/CD pipeline by pushing new code changes and
- verifying that the deployment occurs automatically.



15. Create a GitHub Repository and Implement Version Control Tasks:

- Set up a repository for a team project.
- Create branches for individual modules.
- Merge branches with pull requests after code reviews.
- Resolve conflicts during branch merging.
- Document the workflow using the README file.
- Submit the repository link

